

RPi5 GPIO & WiringPi 실습

실습 교재

GPIO 기초 · LED · 버튼 · 인터럽트 · 부저 · 센서 · PWM · 서보 모터

중앙대학교 전자전기공학부

교재 구성

본 교재는 Raspberry Pi 5의 GPIO(General Purpose I/O) 핀을 WiringPi 라이브러리를 통해 C 언어로 제어하는 방법을 단계별로 설명합니다. 기초 회로 구성부터 LED, 푸시 버튼, 피에조 부저, 조도 센서, PWM, 서보 모터, 초음파 센서, PIR 센서까지 다양한 실습을 포함합니다.

장	주제	주요 내용
1	GPIO 개요 및 WiringPi 설치	GPIO 개념, AMBA 버스, WiringPi 설치, 핀 번호 체계
2	LED 제어	회로 구성, 저항 읽기, wiringPi 함수, LED 점멸 프로그램
3	푸시 버튼 & 인터럽트	폴링 vs 인터럽트, 채터링, ISR 프로그래밍
4	피에조 부저 & 조도 센서	softTone 함수, 음계 배열, 빛 감지 회로
5	PWM 제어 (Software & Hardware)	PWM 원리, LED 밝기 제어, 서보 모터
6	초음파 센서 & PIR 센서	HC-SR04 거리 측정, HC-SR501 동작 감지

참고 문헌

서영진, 사물인터넷을 위한 리눅스 프로그래밍 with 라즈베리파이, 제이펍, 2022.

정원석, 사물인터넷을 위한 라즈베리파이3 활용, 광문각, 2017.

최주호·김재범·정동진, IoT 사물인터넷과 OpenAI 인공지능을 위한 라즈베리파이5 정석 2판, 앤써북, 2024.

제 1장 GPIO 개요 및 WiringPi 설치

1.1 GPIO 란?

GPIO(General Purpose Input/Output)는 '범용 입출력 포트'로, Raspberry Pi나 Arduino 같은 임베디드 보드에서 외부 하드웨어를 연결하고 제어하기 위해 사용되는 다목적 핀 인터페이스입니다. GPIO 핀은 소프트웨어 설정에 따라 입력 또는 출력 방향으로 사용할 수 있으며, 3.3V HIGH 또는 0V(GND) LOW 두 가지 디지털 신호를 처리합니다.

GPIO를 통해 연결할 수 있는 장치는 매우 다양합니다. 입력 장치로는 버튼 스위치, 조도 센서, 온습도 센서, 초음파 센서, PIR 모션 센서 등이 있고, 출력 장치로는 LED, 피에조 부저, 서보 모터, DC 모터 드라이버 등이 있습니다. 이처럼 GPIO는 Raspberry Pi를 단순한 소형 컴퓨터에서 IoT 기기로 변환시켜 주는 핵심 인터페이스입니다.

1.1.1 AMBA 버스와 GPIO

Raspberry Pi 5의 핵심인 BCM2712 SoC는 ARM이 설계한 AMBA(Advanced Microcontroller Bus Architecture) 버스를 통해 GPIO를 비롯한 모든 주변 장치와 통신합니다. AMBA는 다음 세 가지 버스로 구성됩니다.

버스	전체 명칭	역할
AHB	Advanced High-performance Bus	CPU와 고속 메모리 간 고속 데이터 전송을 담당하는 메인 시스템 버스
ASB	Advanced System Bus	AHB의 전신으로, 중간 속도의 시스템 버스. 일부 구형 SoC에서 사용됨
APB	Advanced Peripheral Bus	GPIO, UART, SPI, I2C 등 저속 주변 장치에 연결되는 버스. GPIO는 이 버스를 통해 CPU와 연결됨

GPIO는 APB(Advanced Peripheral Bus)에 SoC 페리퍼럴로 연결되어 있습니다. 소프트웨어에서 GPIO 레지스터에 값을 쓰면 APB를 통해 해당 핀의 전기적 상태가 변경되고, 이를 통해 외부 회로를 제어하거나 외부 신호를 읽을 수 있게 됩니다.

1.1.2 GPIO 핀 구성

Raspberry Pi 5는 40핀 GPIO 헤더를 제공하며, 이 중 실제 GPIO로 사용 가능한 핀이 26개 포함됩니다. 나머지 핀은 전원(5V, 3.3V) 공급 및 GND용입니다. GPIO 핀은 세 가지 번호 체계로 참조할 수 있습니다.

번호 체계	명칭	설명
BCM	Broadcom GPIO 번호	BCM2712 SoC 칩 내부 GPIO 번호. wiringPiSetupGpio() 사용 시 적용. 가장 널리 사용됨

wPi	WiringPi 고유 번호	WiringPi 라이브러리의 자체 핀 번호 체계. wiringPiSetup() 사용 시 적용
Physical	물리 핀 번호	GPIO 헤더의 실제 물리적 위치 번호 (1~40). 보드에서 핀 위치 확인 시 사용

🔗 `gpio readall` 명령어를 실행하면 세 번호 체계를 한눈에 비교할 수 있는 핀 맵이 출력됩니다.

1.1.3 GPIO 제어 원리

소프트웨어에서 GPIO를 제어하는 방식은 크게 출력과 입력으로 나뉩니다.

출력 모드: 소프트웨어가 핀의 방향을 출력(OUTPUT)으로 설정하면, 해당 핀의 전기적 상태를 HIGH(3.3V) 또는 LOW(0V)로 능동적으로 바꿀 수 있습니다. 이 전기 신호로 LED를 켜거나 끄고, 부저를 울리거나 모터를 제어합니다.

입력 모드: 핀의 방향을 입력(INPUT)으로 설정하면, 핀에 인가된 외부 전기 신호를 읽을 수 있습니다. 버튼이 눌렸는지(LOW), 센서가 반응하는지(HIGH) 등을 소프트웨어에서 판단할 수 있습니다.

⚠️ GPIO 핀의 최대 허용 전압은 3.3V입니다. 5V 신호를 직접 인가하면 핀이 영구적으로 손상될 수 있습니다.

1.2 WiringPi 설치

WiringPi는 Raspberry Pi에서 GPIO를 C 언어로 쉽게 제어할 수 있도록 해 주는 라이브러리입니다.

Arduino의 `digitalWrite()`, `digitalRead()` 같은 직관적인 함수를 제공하여 복잡한 레지스터 조작 없이도 하드웨어를 제어할 수 있습니다.

과거에는 apt 패키지 관리자로 간편하게 설치할 수 있었지만, Raspberry Pi OS Bullseye 이후 공식 패키지가 더 이상 지원(deprecated)되지 않습니다. 따라서 현재는 GitHub의 커뮤니티 포크 버전을 소스에서 직접 빌드하는 방법을 사용해야 합니다.

1.2.1 설치 방법 비교

방법	설명	권장 여부
apt install wiringpi	Bullseye 이후 지원 중단된 구형 패키지	✗ 권장하지 않음
공식 .deb 설치	의존성 충돌 문제 다수 발생	✗ 권장하지 않음
GitHub 포크 소스 빌드	최신 RPi5에서도 정상 동작하는 유지보수 버전	☑️ 권장

1.2.2 단계별 설치 절차

GitHub의 커뮤니티 포크 버전을 빌드하여 설치하는 절차입니다. 터미널을 열고 아래 명령어를 순서대로

실행합니다.

1 단계: 필수 패키지 설치

소스 빌드에 필요한 git과 build-essential을 먼저 설치합니다.

```
$ sudo apt update
$ sudo apt install git build-essential
```

2 단계: GitHub 에서 소스 다운로드

WiringPi의 커뮤니티 포크 저장소를 홈 디렉토리에 클론합니다.

```
$ cd ~
$ git clone https://github.com/WiringPi/WiringPi.git
$ cd WiringPi
```

설치할 버전을 확인하고 싶다면 아래 명령어로 최근 커밋 정보를 볼 수 있습니다.

```
$ git log -1
```

3 단계: 빌드 및 설치

빌드 스크립트를 실행합니다. 빌드가 성공적으로 완료되면 gpio 명령어가 /usr/local/bin/gpio 위치에 설치됩니다.

```
$ ./build
```

4 단계: 설치 확인

설치가 완료되면 아래 명령어로 버전과 핀 맵을 확인합니다.

```
$ gpio -v
$ gpio readall
```

gpio -v 명령어를 실행하면 버전(예: gpio version: 3.16)과 함께 보드 모델 정보가 출력됩니다. gpio readall은 40핀 헤더의 전체 핀 번호 매핑과 현재 핀 모드, 값을 표로 출력합니다.

1.2.3 설치 오류 해결

오류 메시지	해결 방법
gpio: command not found	빌드 실패 → ~/WiringPi 디렉토리에서 ./build를 다시 실행. PATH에 /usr/local/bin이 포함되어 있는지 확인
wiringPi: version 2.50 등 낮은 버	오래된 apt 설치 버전이 남아있음 → 위 GitHub 포크 방법으로 재

전	설치
wiringPi: gpio not found	PATH 확인: echo \$PATH로 /usr/local/bin 포함 여부를 확인하고, export PATH=\$PATH:/usr/local/bin을 .bashrc에 추가

1.2.4 기본 gpio 명령어 사용법

WiringPi 설치 후 gpio 셸 명령어로 프로그램 작성 없이 GPIO 핀을 직접 제어할 수 있습니다. -g 옵션은 BCM 번호 체계를 사용하도록 지정합니다.

```
$ gpio -g mode 18 out      # GPIO18 핀을 출력 모드로 설정
$ gpio -g write 18 1      # GPIO18 핀을 HIGH(3.3V)로 출력
$ gpio -g write 18 0      # GPIO18 핀을 LOW(0V)로 출력
$ gpio -g mode 24 in      # GPIO24 핀을 입력 모드로 설정
$ gpio -g read 24         # GPIO24 핀의 현재 값 읽기 (0 또는 1 출력)
```

🔗 참고: <https://chatgpt.com/share/69798c15-9c0c-8004-a0b7-cdc36b6aa02>

1.3 WiringPi 핵심 API

WiringPi 라이브러리는 GPIO 제어에 필요한 초기화, 핀 모드 설정, 디지털 입출력, 지연 함수 등을 제공합니다. 모든 WiringPi 프로그램은 가장 먼저 초기화 함수를 호출해야 합니다.

1.3.1 초기화 함수

함수	설명
wiringPiSetup()	핀 번호를 WiringPi 고유 번호(wPi)로 사용. 이후 함수에서 wPi 번호로 핀 참조
wiringPiSetupGpio()	핀 번호를 BCM 번호로 사용. 실제 GPIO 번호와 동일하게 참조 가능하여 회로도화 맞추기 편리함

1.3.2 핀 제어 함수

함수 원형	설명
void pinMode(int pin, int mode)	핀의 방향을 설정. mode: INPUT(입력), OUTPUT(출력), PWM_OUTPUT(PWM 출력)
void digitalWrite(int pin, int value)	핀에 디지털 값 출력. value: HIGH(1) 또는 LOW(0). OUTPUT 모드로 설정된 핀에 사용
int digitalRead(int pin)	핀의 현재 디지털 값을 읽어 반환. 반환값: HIGH(1) 또는 LOW(0). INPUT 모드 핀에 사용
void delay(unsigned int ms)	밀리초(1/1000초) 단위로 프로그램 실행을 일시 중지
void delayMicroseconds(unsigned int us)	마이크로초(1/1,000,000초) 단위로 지연. 정밀한 타이밍 제어에 사용 (예: 초음파 펄스)

🔗 wiringPiSetup()을 사용하는 경우 wPi 번호, wiringPiSetupGpio()를 사용하는 경우 BCM 번호로 핀을 참조함

니다. 같은 프로그램 내에서 두 함수를 혼용하면 안 됩니다.

제 2장 LED 제어

2.1 회로 구성

LED를 Raspberry Pi 5에 연결하여 소프트웨어로 제어하려면 브레드보드(Breadboard)를 사용합니다. 브레드보드는 납땜 없이 전자 부품을 빠르게 연결하고 테스트할 수 있는 실습용 기관입니다. 내부적으로 가로 방향의 구멍들이 서로 연결되어 있어 부품을 꽂기만 하면 전기적으로 이어집니다.

2.1.1 LED 극성 확인

LED(Light Emitting Diode)는 극성이 있는 부품으로, 올바른 방향으로 연결해야 동작합니다. 두 리드(단자) 중 긴 쪽이 양극(Anode, +)이고 짧은 쪽이 음극(Cathode, -)입니다. 양극은 GPIO 출력 핀 쪽으로, 음극은 저항을 거쳐 GND 쪽으로 연결합니다.

⚠ LED를 저항 없이 직접 GPIO에 연결하면 과전류가 흘러 LED와 GPIO 핀이 모두 손상됩니다. 반드시 220Ω 이상의 전류 제한 저항을 직렬로 연결하십시오.

2.1.2 저항값 읽기 (컬러 코드)

저항의 색띠는 저항값을 나타내는 코드입니다. 4띠 저항의 경우 처음 두 띠가 유효 숫자, 세 번째 띠가 배수(곱하기 값), 네 번째 띠가 허용 오차를 나타냅니다. 색상별 숫자는 검정=0, 갈색=1, 빨강=2, 주황=3, 노랑=4, 초록=5, 파랑=6, 보라=7, 회색=8, 흰색=9입니다.

예를 들어 '갈-검-갈-금' 띠를 가진 저항은 갈(1) + 검(0) × 갈(×10) = 100Ω, 허용 오차 ±5%(금) 입니다.

LED 제어에 주로 사용되는 220Ω 저항은 '빨-빨-갈(갈색)' 또는 '빨-빨-검-검' 등의 색띠로 확인할 수 있습니다.

2.1.3 연결 방법

아래 순서로 LED와 Raspberry Pi를 브레드보드를 통해 연결합니다.

1. LED의 긴 다리(양극, Anode)를 브레드보드에 꽂습니다.
2. LED 양극과 같은 행에 220Ω 저항의 한쪽 끝을 꽂습니다.
3. 저항의 다른 쪽 끝에서 점퍼 와이어를 사용하여 GPIO 18번 핀(BCM)에 연결합니다.
4. LED의 짧은 다리(음극, Cathode)에서 점퍼 와이어를 GND 핀(예: 6번 물리 핀)에 연결합니다.

🔗 GPIO 18번(BCM)은 WiringPi 번호로는 1번에 해당합니다. `gpio readall` 출력에서 BCM 열의 18과 wPi 열의 1이 같은 행에 있음을 확인할 수 있습니다.

2.2 WiringPi LED 제어 프로그램

회로가 준비되었으면 C 언어로 LED를 5번 점멸시키는 프로그램을 작성합니다. 이 프로그램은 실행 시 GPIO 번호를 인자(argv[1])로 받아 범용적으로 사용할 수 있도록 설계되어 있습니다.

2.2.1 소스 코드 (wiringled.c)

```
#include <wiringPi.h>    /* WiringPi 라이브러리 헤더 */
#include <stdio.h>
#include <stdlib.h>       /* atoi() 함수 */

int ledControl(int gpio)
{
    int i;

    pinMode(gpio, OUTPUT);    /* 지정 핀을 출력 모드로 설정 */

    for (i = 0; i < 5; i++) {
        digitalWrite(gpio, HIGH); /* HIGH 출력: LED 켜기 */
        delay(1000);             /* 1초 대기 */
        digitalWrite(gpio, LOW);  /* LOW 출력: LED 끄기 */
        delay(1000);
    }

    return 0;
}

int main(int argc, char **argv)
{
    int gno;

    if (argc < 2) {
        printf("Usage: %s GPIO_NO\n", argv[0]);
        return -1;
    }

    gno = atoi(argv[1]);    /* 문자열 인자를 정수로 변환 */

    wiringPiSetup();        /* WiringPi 초기화 (wPi 번호 사용) */
    ledControl(gno);

    return 0;
}
```

2.2.2 컴파일 및 실행

-lwiringPi 옵션은 WiringPi 라이브러리를 링크하도록 지시합니다. 모든 WiringPi 프로그램 컴파일 시 반드시 포함해야 합니다.

```
$ gcc -o wiringled wiringled.c -lwiringPi
$ ./wiringled 1          # wPi 번호 1 = GPIO18
```

2.2.3 코드 해설

wiringPiSetup()은 WiringPi 고유 번호(wPi) 체계로 초기화하므로, 이후 모든 핀 번호는 wPi 기준입니다. wPi 번호 1은 BCM 18번과 동일합니다. BCM 번호를 그대로 사용하려면 wiringPiSetup() 대신 wiringPiSetupGpio()를 호출해야 합니다.

atoi(argv[1])는 커맨드라인에서 문자열로 전달된 GPIO 번호를 정수로 변환합니다. 이 방식을 사용하면 컴파일 없이 다른 핀 번호를 인자로 줘서 테스트할 수 있습니다.

제 3장 푸시 버튼 & 인터럽트

3.1 푸시 버튼 (Tack Switch)

푸시 버튼(또는 Tack Switch)은 누르는 동안 회로가 연결되고, 손을 떼면 다시 열리는 순간 접점 스위치입니다. 풀업(Pull-up) 저항을 사용하여 연결하면 버튼을 누르지 않았을 때 HIGH, 누른 상태에서 LOW 신호가 읽힙니다. 반대로 풀다운(Pull-down) 구성에서는 반대 동작을 합니다.

버튼은 GPIO 24번(BCM)에 연결하고, LED는 GPIO 18번(BCM)에 연결합니다. 내부 풀업 저항을 활성화하거나 외부 풀업 저항(10kΩ)을 사용하면 버튼이 눌리지 않은 상태에서 3.3V(HIGH)가 유지됩니다.

3.1.1 폴링 방식 푸시 버튼 프로그램 (wiringswitch.c)

폴링(Polling)은 소프트웨어가 주기적으로 핀 상태를 직접 확인하는 방식입니다. 무한 루프(for(;;)) 안에서 digitalRead()를 반복 호출하여 버튼 눌림을 감지합니다.

```
#include <wiringPi.h>
#include <stdio.h>

#define SW 5    /* wPi 5 = GPIO24: 버튼 핀 */
#define LED 1   /* wPi 1 = GPIO18: LED 핀 */

int switchControl()
{
    pinMode(SW, INPUT);    /* 버튼 핀: 입력 모드 */
    pinMode(LED, OUTPUT);

    for (;;) {             /* 무한 루프: 계속 상태 확인 */
        if (digitalRead(SW) == LOW) { /* 버튼 눌림(LOW 감지) */
            digitalWrite(LED, HIGH); /* LED 켜기 */
            delay(1000);
            digitalWrite(LED, LOW);  /* LED 끄기 */
        }
        delay(10); /* CPU 점유율 완화를 위한 짧은 지연 */
    }

    return 0;
}

int main(int argc, char **argv)
{
    wiringPiSetup();
    switchControl();
    return 0;
}
```

```
$ gcc -o wiringswitch wiringswitch.c -lwiringPi
$ ./wiringswitch
```

3.2 채터링 (Chattering)

버튼을 누르거나 떼는 순간, 기계적 접점이 완전히 붙거나 떨어지기 전 짧은 시간 동안 빠르게 ON/OFF를 반복하는 전기적 잡음이 발생합니다. 이를 채터링(Chattering) 또는 바운싱(Bouncing)이라고 합니다. 수 마이크로초~수 밀리초 동안 발생하는 이 잡음 때문에, 버튼을 한 번 눌렀음에도 소프트웨어에서는 여러 번 눌린 것으로 인식할 수 있습니다.

채터링은 하드웨어(커패시터를 달아 고주파 제거)와 소프트웨어(버튼 이벤트 발생 후 수~수십 밀리초 대기) 두 가지 방법으로 처리할 수 있습니다. 위 폴링 코드의 `delay(10)`은 소프트웨어 방식의 간단한 채터링 방지 역할도 합니다.

3.3 폴링 vs 인터럽트

버튼 입력을 감지하는 방법은 폴링(Polling)과 인터럽트(Interrupt) 두 가지가 있습니다. 각 방식의 동작 원리와 장단점을 이해하면 상황에 맞는 설계를 할 수 있습니다.

구분	폴링 (Polling)	인터럽트 (Interrupt)
동작 방식	CPU가 루프를 돌며 주기적으로 핀 상태를 직접 확인	이벤트(엣지 변화) 발생 시 하드웨어가 CPU에 신호를 보내 ISR 자동 호출
응답성	루프 주기에 따라 지연 발생 가능	이벤트 즉시 처리, 빠른 응답
CPU 사용률	루프가 계속 돌아 CPU를 지속 점유	이벤트 없을 때 CPU를 다른 작업에 활용 가능
구현 복잡도	단순 (루프 + 조건문)	상대적으로 복잡 (ISR 함수 등록 필요)
적합한 상황	간단한 구현, 이벤트가 드물지 않을 때	정밀한 타이밍, 이벤트가 드물거나 즉각 대응이 필요할 때

3.4 인터럽트 방식 프로그램

인터럽트 방식에서는 GPIO 핀의 전기 신호 변화(Rising edge 또는 Falling edge)를 감지하면 CPU가 현재 작업을 잠시 중단하고 미리 등록된 ISR(Interrupt Service Routine) 함수를 자동으로 실행합니다. ISR 실행이 끝나면 중단된 작업으로 복귀합니다.

Falling edge: 신호가 HIGH → LOW로 하강하는 순간. 버튼을 누르는 순간에 해당

Rising edge: 신호가 LOW → HIGH로 상승하는 순간. 버튼에서 손을 떼는 순간에 해당

INT_EDGE_BOTH: Rising과 Falling 모두 감지

3.4.1 ISR 등록 함수

`wiringPiISR()` 함수를 사용하여 특정 핀의 엣지 변화에 반응할 ISR을 등록합니다.

```
int wiringPiISR(int pin, int mode, void (*function)(void))
```

pin은 핀 번호, mode는 감지할 엣지 종류(INT_EDGE_FALLING / INT_EDGE_RISING / INT_EDGE_BOTH), function은 이벤트 발생 시 호출할 ISR 함수의 포인터입니다.

3.4.2 인터럽트 프로그램 예제 (wiringint2.c)

아래 프로그램은 버튼 눌림/떼기 이벤트를 인터럽트로 감지하면서, 메인 루프는 매초 경과 시간을 출력합니다. 인터럽트를 사용하면 메인 루프에서 버튼을 계속 확인할 필요 없이 다른 작업(여기서는 시간 출력)을 동시에 수행할 수 있습니다.

```
#include <stdio.h>
#include <wiringPi.h>

#define LED 18    /* BCM 18 */
#define KEY 24    /* BCM 24 */

void isr_key()                /* ISR 함수 */
{
    int val = digitalRead(KEY);

    if (val == LOW) {          /* 버튼 눌림 */
        printf("Key is pressed \r\n");
        digitalWrite(LED, 1);
    } else {                   /* 버튼 떼기 */
        printf("Key is released \r\n");
        digitalWrite(LED, 0);
    }
}

void main(void)
{
    unsigned int sec = 0;

    wiringPiSetupGpio();        /* BCM 번호 체계 사용 */
    pinMode(LED, OUTPUT);
    pinMode(KEY, INPUT);
    wiringPiISR(KEY, INT_EDGE_BOTH, isr_key); /* ISR 등록 */

    while (1) {                /* 메인 루프: 타이머 출력 */
        printf("%d sec. \r\n", sec);
        sec++;
        delay(1000);
    }
}
```

```
$ gcc -o wiringint2 wiringint2.c -lwiringPi
$ ./wiringint2
```

프로그램 실행 중 버튼을 누르면 메인 루프가 멈추지 않고 계속 초를 세는 동시에 버튼 이벤트가 즉시 처리됩니다. 이것이 인터럽트 방식의 핵심 장점입니다.

제 4장 피에조 부저 & 조도 센서

4.1 피에조 부저 (소프트웨어 PWM 음악)

피에조 부저는 전압을 가하면 내부 압전 소자가 진동하여 소리를 내는 출력 장치입니다. 다양한 주파수의 신호를 빠르게 ON/OFF 반복하면 그 주파수에 해당하는 음계를 낼 수 있습니다. WiringPi의 `softTone` 라이브러리를 사용하면 소프트웨어 타이머로 원하는 주파수를 쉽게 생성할 수 있습니다.

부저는 GPIO 25번(BCM, wPi 번호 6)에 연결합니다. 부저의 + 단자를 GPIO 핀에, - 단자를 GND에 연결합니다.

4.1.1 음계와 주파수

음악은 각 음계(도, 레, 미, 파, 솔, 라, 시)가 특정 주파수에 대응합니다. 아래는 4옥타브 기준 주요 음계의 주파수입니다.

음계	도(C4)	레(D4)	미(E4)	파(F4)	솔(G4)	라(A4)	시(B4)
주파수 (Hz)	261.63	293.66	329.63	349.23	391.00	440.00	493.88

4.1.2 소프트 톤 음악 프로그램 (wiringmusic.c)

```
#include <wiringPi.h>
#include <softTone.h>      /* 소프트 톤 라이브러리 */

#define SPKR 6      /* wPi 6 = GPIO25: 부저 핀 */
#define TOTAL 32    /* 학교종 음 개수 */

/* 학교종 음계 주파수 배열 (0 = 침표) */
int notes[] = {
    391, 391, 440, 440, 391, 391, 329.63, 329.63,
    391, 391, 329.63, 329.63, 293.66, 293.66, 293.66, 0,
    391, 391, 440, 440, 391, 391, 329.63, 329.63,
    391, 329.63, 293.66, 329.63, 261.63, 261.63, 261.63, 0
};

int musicPlay()
{
    int i;
    softToneCreate(SPKR);      /* 소프트 톤 GPIO 초기화 */
    for (i = 0; i < TOTAL; ++i) {
        softToneWrite(SPKR, notes[i]); /* 해당 주파수로 소리 출력 */
        delay(280);                /* 음 길이만큼 대기 */
    }
    return 0;
}

int main()
{
    wiringPiSetup();
```

```

    musicPlay();
    return 0;
}

```

```

$ gcc -o wiringmusic wiringmusic.c -lwiringPi
$ ./wiringmusic

```

`softToneCreate()`는 지정된 핀에 소프트웨어 타이머 기반의 음 출력 채널을 생성합니다. `softToneWrite()`에 주파수 값을 전달하면 해당 주파수로 신호를 출력합니다. 주파수 0을 전달하면 소리가 멈춥니다(선택).

4.2 조도(빛 감지) 센서

조도 센서(CdS 셀 또는 광의존성 저항, LDR)는 빛의 세기에 따라 저항값이 변하는 소자입니다. 빛이 강할 때 저항이 낮아지고, 어두울 때 저항이 높아집니다. Raspberry Pi의 GPIO는 디지털 신호만 읽을 수 있으므로, 특정 임계값을 기준으로 HIGH/LOW로 판단합니다. 회로에서 조도 센서와 고정 저항을 분압기(전압 분배기) 형태로 연결하면 빛의 세기에 따라 GPIO 핀에 읽히는 전압이 달라집니다.

조도 센서는 GPIO 17번(BCM, wPi 번호 0)에, 버튼은 GPIO 24번(BCM, wPi 5)에, LED는 GPIO 18번(BCM, wPi 1)에 연결합니다.

4.2.1 조도 센서 제어 프로그램 (wiringlight.c)

```

#include <wiringPi.h>
#include <stdio.h>

#define SW 5    /* wPi 5 = GPIO24 */
#define CDS 0   /* wPi 0 = GPIO17: 조도 센서 */
#define LED 1   /* wPi 1 = GPIO18: LED */

int cdsControl()
{
    pinMode(SW, INPUT);
    pinMode(CDS, INPUT); /* 조도 센서 핀 입력 모드 */
    pinMode(LED, OUTPUT);

    for (;;) {
        if (digitalRead(CDS) == HIGH) { /* 빛 감지 시 HIGH */
            digitalWrite(LED, HIGH); /* LED 켜기 */
            delay(1000);
            digitalWrite(LED, LOW);
        }
    }
    return 0;
}

int main()
{
    wiringPiSetup();
    cdsControl();
    return 0;
}

```

```
$ gcc -o wiringlight wiringlight.c -lwiringPi
$ ./wiringlight
```

프로그램이 실행된 상태에서 손으로 센서를 가리면 빛이 차단되어 LED가 꺼지고, 빛을 비추면 LED가 1초간 점멸합니다.

제 5장 PWM 제어 (Software & Hardware)

5.1 PWM 이란?

PWM(Pulse Width Modulation, 펄스 폭 변조)은 디지털 신호(HIGH/LOW만 출력 가능)를 이용하여 아날로그적인 효과를 내는 기법입니다. 일정한 주기(Period) 안에서 HIGH 상태를 유지하는 시간의 비율인 듀티 사이클(Duty Cycle)을 조절하면 평균 전압을 변화시킬 수 있습니다.

예를 들어 듀티 사이클이 50%면 HIGH와 LOW가 절반씩이므로 평균 전압은 $1.65V(3.3V \times 0.5)$ 가 됩니다. LED에 이 신호를 적용하면 완전히 켜진 것보다 어둡게 보이며, 듀티 사이클을 0~100%로 변화시키면 LED 밝기를 부드럽게 조절할 수 있습니다.

5.2 Software PWM vs Hardware PWM

구분	Software PWM	Hardware PWM
초기화	<code>softPwmCreate()</code>	<code>pinMode(pin, PWM_OUTPUT)</code>
값 변경	<code>softPwmWrite()</code>	<code>pwmWrite()</code>
주파수	설정 불가 (약 100Hz 고정)	<code>pwmSetClock()</code> + <code>pwmSetRange()</code> 로 자유롭게 설정
CPU 부하	높음 (SW 타이머로 생성)	낮음 (전용 하드웨어 회로가 처리)
안정성	보통 (다른 작업 시 지터 발생)	매우 안정 (하드웨어가 독립적으로 동작)
사용 가능 핀	거의 모든 GPIO 핀	GPIO 12, 13, 18, 19만 가능
적합한 용도	LED 밝기, 간단한 부저	서보 모터, 고정밀 부저, 정밀 PWM

5.2.1 Hardware PWM 채널

Raspberry Pi 5는 두 개의 하드웨어 PWM 채널을 제공합니다. 두 채널을 동시에 독립적으로 제어하려면 각 채널에서 하나씩 핀을 선택해야 합니다.

채널	사용 가능 GPIO (BCM)	동시 사용 시 조합 예
PWM0	GPIO 12, GPIO 18	GPIO 18 (PWM0) + GPIO 13 (PWM1)을 동시 사용 가능
PWM1	GPIO 13, GPIO 19	두 채널에서 각 하나씩 선택

5.3 Hardware PWM LED 밝기 제어

Hardware PWM을 사용하면 LED의 밝기를 CPU 부하 없이 안정적으로 제어할 수 있습니다. GPIO 18번 (BCM, PWM0 채널)에 LED와 220Ω 저항을 연결합니다.

pwmSetClock()과 pwmSetRange()로 PWM 주파수를 설정합니다. RPi 5의 PWM 기준 클럭은 19.2MHz이며 주파수 계산식은 다음과 같습니다.

```
PWM 주파수 = 19.2MHz / (Clock * Range)
예: 19.2MHz / (75 * 255) ≈ 1003Hz (약 1kHz)
```

5.3.1 Hardware PWM LED 프로그램 (led_pwm.c)

```
#include <wiringPi.h>
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

int gpio_pin;

void cleanup(int sig)          /* Ctrl+C 처리: GPIO 정리 */
{
    pwmWrite(gpio_pin, 0);
    pinMode(gpio_pin, INPUT);
    printf("\nPWM 종료 및 GPIO 정리\n");
    exit(0);
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("Usage: %s GPIO_NO(BCM)\n", argv[0]);
        return 1;
    }

    gpio_pin = atoi(argv[1]);

    if (wiringPiSetupGpio() == -1) { /* BCM 모드 초기화 */
        printf("wiringPi 초기화 실패\n");
        return 1;
    }

    signal(SIGINT, cleanup);
    pinMode(gpio_pin, PWM_OUTPUT);

    pwmSetMode(PWM_MODE_MS); /* Mark-Space 모드 */
    pwmSetRange(255);        /* 0~255 범위 */
    pwmSetClock(75);         /* 약 1kHz */

    for (int i = 0; i < 10000; i++) {
        int duty = i % 256; /* 0→255→0→... 반복 */
        pwmWrite(gpio_pin, duty);
        delay(5);
    }

    cleanup(0);
    return 0;
}
```

```
$ gcc -o led_pwm led_pwm.c -lwiringPi
$ ./led_pwm 18          # BCM 18번 핀
```

5.4 PWM 서보 모터 제어

서보 모터는 PWM 신호의 펄스 폭에 따라 회전 각도가 결정되는 정밀 액추에이터입니다. 일반 서보 모터는 50Hz(주기 20ms)의 PWM 신호를 입력받으며, 펄스 폭에 따라 0도에서 180도까지 회전합니다.

각도	펄스 폭	듀티 사이클	pwmWrite 값 (Range=2000)
0도	0.5ms	2.5%	50
90도	1.5ms	7.5%	150
180도	2.5ms	12.5%	250

⚠ RPi5의 GPIO 5V 핀(물리 2번 또는 4번)으로 서보 모터를 직접 구동하면 과전류로 보드가 손상됩니다. 반드시 외부 5V 전원(별도 어댑터, 최소 2A)으로 서보의 VCC를 공급하고, GND는 RPi와 공통 접지하십시오. PWM 신호선만 GPIO 핀에 연결하며, 신호선에 1kΩ 직렬 저항 추가를 권장합니다.

5.4.1 서보 모터 PWM 주파수 설정

서보 모터에는 50Hz PWM이 필요합니다. pwmSetClock과 pwmSetRange를 다음과 같이 설정합니다.

```
pwmSetMode(PWM_MODE_MS);
pwmSetRange(2000);
pwmSetClock(192);
/* 19.2MHz / (192 * 2000) = 50Hz */
```

5.4.2 서보 모터 프로그램 (servo_pwm.c)

```
#include <wiringPi.h>
#include <stdio.h>
#include <signal.h>

#define SERVO_PIN 18 /* BCM 18 (PWM0 채널) */

void cleanup(int sig)
{
    pwmWrite(SERVO_PIN, 0);
    pinMode(SERVO_PIN, INPUT);
    printf("\n서보 종료 및 GPIO 정리\n");
    exit(0);
}

int main(void)
{
    if (wiringPiSetupGpio() == -1) return 1;

    signal(SIGINT, cleanup);
    pinMode(SERVO_PIN, PWM_OUTPUT);

    pwmSetMode(PWM_MODE_MS);
    pwmSetRange(2000);
    pwmSetClock(192); /* 50Hz: 19.2M/(192*2000) */

    while (1) {
        pwmWrite(SERVO_PIN, 150); /* 1.5ms → 90도 */
    }
}
```

```
    delay(1000);  
    pwmWrite(SERVO_PIN, 250); /* 2.5ms → 180도 */  
    delay(1000);  
    pwmWrite(SERVO_PIN, 50); /* 0.5ms → 0도 */  
    delay(1000);  
}  
return 0;  
}
```

```
$ gcc -o servo_pwm servo_pwm.c -lwiringPi  
$ ./servo_pwm
```

제 6장 초음파 센서 & PIR 센서

6.1 초음파 센서 (HC-SR04)

HC-SR04는 초음파(40kHz)를 발사하고 반사되어 돌아오는 시간을 측정하여 물체까지의 거리를 계산하는 거리 측정 센서입니다. 측정 범위는 약 2cm~400cm이며, 정밀도는 약 3mm입니다.

6.1.1 동작 원리

HC-SR04는 TRIG와 ECHO 두 개의 제어 핀을 사용합니다. 동작 순서는 다음과 같습니다.

1. MCU가 TRIG 핀에 최소 10 μ s 동안 HIGH 신호를 출력합니다.
2. 센서 내부에서 40kHz 초음파 펄스 8개를 발사합니다.
3. 초음파가 물체에 반사되어 돌아오면 ECHO 핀이 HIGH 상태로 올라갑니다.
4. MCU는 ECHO 핀이 HIGH인 시간(왕복 시간)을 측정합니다.
5. 거리 = 왕복 시간 $\times$ 음속(34300 cm/s) / 2 로 계산합니다.

TRIG 핀은 GPIO 23번(BCM), ECHO 핀은 GPIO 24번(BCM)에 연결합니다.

⚠ HC-SR04의 ECHO 핀은 5V 신호를 출력합니다. Raspberry Pi의 GPIO는 3.3V 입력 허용이므로 전압 분배기(저항 분압) 또는 레벨 시프터를 ECHO 선에 삽입하지 않으면 GPIO가 손상될 수 있습니다.

6.1.2 초음파 센서 프로그램 (HC_SR04.c)

```
#include <wiringPi.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <signal.h>

#define TRIG 23    /* BCM 23: 트리거 핀 */
#define ECHO 24    /* BCM 24: 에코 핀 */

void cleanup(int sig)
{
    digitalWrite(TRIG, LOW);
    printf("\nMeasurement stopped by User\n");
    exit(0);
}

double getTime()    /* 현재 시간을 초 단위(double)로 반환 */
{
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}
```

```

int main(void)
{
    if (wiringPiSetupGpio() == -1) return 1;
    signal(SIGINT, cleanup);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    digitalWrite(TRIG, LOW);
    printf("Waiting for sensor to settle\n");
    delay(2000);          /* 센서 안정화 2초 대기 */

    while (1) {
        /* 10µs TRIG 펄스 생성 */
        digitalWrite(TRIG, HIGH);
        delayMicroseconds(10);
        digitalWrite(TRIG, LOW);

        while (digitalRead(ECHO) == LOW); /* ECHO 상승 대기 */
        double start = getTime();

        while (digitalRead(ECHO) == HIGH); /* ECHO 하강 대기 */
        double stop = getTime();

        double elapsed = stop - start;
        double distance = elapsed * 34300 / 2; /* 거리(cm) 계산 */

        printf("Distance : %.1f cm\n", distance);
        delay(400);
    }
    return 0;
}

```

```

$ gcc -o HC_SR04 HC_SR04.c -lwiringPi
$ ./HC_SR04

```

프로그램이 실행되면 2초의 안정화 대기 후 물체까지의 거리를 0.4초 간격으로 cm 단위로 출력합니다. 손을 센서 앞에서 움직이면 거리 값이 변하는 것을 실시간으로 확인할 수 있습니다. Ctrl+C로 종료합니다.

6.2 PIR 모션 센서 (HC-SR501)

PIR(Passive Infrared, 수동 적외선) 센서는 사람의 체온이 방출하는 적외선을 감지하여 움직임을 탐지하는 센서입니다. 조명, 보안 시스템, 자동 도어 등에 널리 사용됩니다. HC-SR501은 감지 범위 내에서 적외선 변화(사람이 움직임)가 있을 때 OUT 핀에 HIGH 신호를 출력하고, 변화가 없으면 LOW를 유지합니다.

6.2.1 HC-SR501 특징

HC-SR501 센서에는 두 개의 조절 나사가 있습니다. 하나는 감도(Sensitivity) 조절로 감지 거리(3~7m)를 설정하고, 다른 하나는 지연 시간(Time Delay) 조절로 동작 감지 후 HIGH 신호를 유지하는 시간(5초~300초)을 설정합니다. 전원 인가 후 약 5~60초의 워밍업(안정화) 시간이 필요합니다.

RED LED(빨간)는 GPIO 20번(BCM), YELLOW LED(노란)는 GPIO 21번(BCM), PIR 센서 신호는 GPIO

17번(BCM)에 연결합니다.

6.2.2 PIR 센서 프로그램 (HC_SR501.c)

```
#include <wiringPi.h>
#include <stdio.h>
#include <signal.h>

#define LED_R  20  /* BCM 20: 빨간 LED (정지 상태 표시) */
#define LED_Y  21  /* BCM 21: 노란 LED (동작 감지 표시) */
#define SENSOR 17  /* BCM 17: PIR 센서 출력 */

void cleanup(int sig)
{
    digitalWrite(LED_R, LOW);
    digitalWrite(LED_Y, LOW);
    printf("\nStopped by User\n");
    exit(0);
}

int main(void)
{
    if (wiringPiSetupGpio() == -1) return 1;
    signal(SIGINT, cleanup);

    pinMode(LED_R,  OUTPUT);
    pinMode(LED_Y,  OUTPUT);
    pinMode(SENSOR, INPUT);

    printf("PIR Ready . . . .\n");
    delay(5000); /* PIR 안정화 5초 대기 */

    while (1) {
        if (digitalRead(SENSOR) == HIGH) { /* 동작 감지 */
            digitalWrite(LED_Y, HIGH);
            digitalWrite(LED_R, LOW);
            printf("Motion Detected !\n");
        } else { /* 정지 상태 */
            digitalWrite(LED_R, HIGH);
            digitalWrite(LED_Y, LOW);
            printf("NO Motion !\n");
        }
        delay(200);
    }
    return 0;
}
```

```
$ gcc -o HC_SR501 HC_SR501.c -lwiringPi
$ ./HC_SR501
```

프로그램 실행 후 5초 안정화 시간이 지나면 센서가 동작을 감지할 때 노란 LED가 켜지고 'Motion Detected!'가 출력됩니다. 움직임이 없으면 빨간 LED가 켜지고 'NO Motion!'이 출력됩니다.

 PIR 센서는 전원 인가 직후 잘못된 HIGH 신호를 낼 수 있습니다. 이 때문에 프로그램에서 delay(5000)으로 5초 안정화 시간을 두고 있습니다. 실제 환경에서는 더 긴 안정화 시간이 필요할 수도 있습니다.

참고 문헌

서영진, 사물인터넷을 위한 리눅스 프로그래밍 with 라즈베리파이, 제이펍, 2022 — Chapter 3. 라즈베리파이와 GPIO: 하드웨어 제어

정원석, 사물인터넷을 위한 라즈베리파이3 활용, 광문각, 2017 — Chapter 15. 하드웨어 제어

최주호·김재범·정동진, IoT 사물인터넷과 OpenAI 인공지능을 위한 라즈베리파이5 정석 2판, 앤써북, 2024 — Ch05. 라즈베리파이 GPIO 및 센서 활용하기